Bachelor's diploma thesis

in the field of study Computer Science and Information Systems

Design and Implementation of a Knowledge-Retrieval-Supported Application for Creating Laboratory Assignments

Larisa Salamatova

student record book number 330353

Stanislaw Konariew

student record book number 335945

Supervisor: Dr hab. inż. Jerzy Balicki, prof. PW

WARSAW 2027

Abstract

This thesis concerns the design, implementation, and evaluation of a configurable web application for creating laboratory assignments for academic courses using Retrieval-Augmented Generation (RAG). The application is intended to generate structured, evidence-grounded assignments from approved course materials while allowing instructors to specify pedagogical requirements, including topic, learning outcomes, difficulty level, course stage, target competencies, expected duration, and assignment format.

The proposed solution uses closed, course-specific knowledge bases and does not use web search. It distinguishes between a primary knowledge base containing authoritative materials for the target course and a supplementary knowledge base containing approved materials from related or earlier courses. Mandatory technical requirements, course-specific tools, learning outcomes, and assessment criteria are required to be supported by primary-course materials. Supplementary materials may provide contextual explanations, prerequisites, hints, and optional extensions, but may not override primary sources.

The application is evaluated using Operating Systems course materials as a case study. A reproducible comparative evaluation examines baseline vector RAG, Hybrid RAG, LightRAG, and GraphRAG under a shared corpus, generation model, prompt template, output schema, validation procedure, held-out benchmark, and evaluation protocol. The evaluation considers retrieval relevance, evidence coverage, citation correctness, factual correctness, learning-outcome alignment, expert-rated usefulness, validation performance, latency, indexing time, storage use, token usage, and estimated inference cost.

The expected outcome is a tested and deployable instructor-facing application with document and metadata management, corpus preparation, assignment configuration, evidence inspection, editing, regeneration, and export capabilities. The work also delivers reproducible benchmark materials, experiment configurations, automated tests, evaluation rubrics, scripts, and aggregated results. The default retrieval architecture is selected using a predefined decision rule that prioritizes grounding, factual correctness, and learning-outcome alignment while considering operational cost.

Keywords: Retrieval-Augmented Generation, large language models, information retrieval, laboratory assignment generation, evidence grounding, source traceability, educational technology, knowledge graphs.

Streszczenie

Niniejsza praca dyplomowa dotyczy zaprojektowania, implementacji i oceny konfigurowalnej aplikacji internetowej wspierającej tworzenie zadań laboratoryjnych dla kursów akademickich z wykorzystaniem generowania wspomaganego wyszukiwaniem informacji (Retrieval-Augmented Generation, RAG). Aplikacja ma generować ustrukturyzowane zadania oparte na dowodach pochodzących z zatwierdzonych materiałów dydaktycznych, z uwzględnieniem konfigurowalnych wymagań pedagogicznych, takich jak temat, efekty uczenia się, poziom trudności, etap kursu, docelowe kompetencje, przewidywany czas realizacji oraz format zadania.

Proponowane rozwiązanie wykorzystuje zamknięte, specyficzne dla kursu bazy wiedzy i nie korzysta z wyszukiwania w sieci Internet. Rozróżnia ono podstawową bazę wiedzy, zawierającą autorytatywne materiały dla kursu docelowego, oraz uzupełniającą bazę wiedzy, obejmującą zatwierdzone materiały z kursów powiązanych lub wcześniejszych. Obowiązkowe wymagania techniczne, narzędzia specyficzne dla kursu, efekty uczenia się oraz kryteria oceny muszą być potwierdzone materiałami podstawowymi. Materiały uzupełniające mogą być wykorzystywane do przedstawiania wyjaśnień kontekstowych, wymagań wstępnych, wskazówek oraz opcjonalnych rozszerzeń, lecz nie mogą zastępować materiałów podstawowych.

Aplikacja zostanie oceniona na podstawie materiałów z kursu Systemy Operacyjne. Odtwarzalna ocena porównawcza obejmie bazowy wariant wektorowego RAG, Hybrid RAG, LightRAG oraz GraphRAG, stosowane z wykorzystaniem wspólnego korpusu, modelu generatywnego, szablonu promptu, schematu wyjściowego, procedury walidacji, wydzielonego zbioru testowego oraz protokołu ewaluacyjnego. Ocena obejmie trafność wyszukiwania, pokrycie dowodami, poprawność cytowań, poprawność faktograficzną, zgodność z efektami uczenia się, użyteczność ocenianą przez ekspertów, skuteczność mechanizmu walidacji, opóźnienie, czas indeksowania, zużycie pamięci, wykorzystanie tokenów oraz szacowany koszt wnioskowania.

Oczekiwanym rezultatem będzie przetestowana i gotowa do wdrożenia aplikacja internetowa dla prowadzących zajęcia, oferująca zarządzanie dokumentami i metadanymi, przygotowanie korpusu, konfigurację zadań, kontrolę dowodów, edycję, ponowne generowanie oraz eksport. Praca dostarczy również odtwarzalne materiały benchmarkowe, konfiguracje eksperymentów, testy automatyczne, rubryki ewaluacyjne, skrypty oraz zagregowane wyniki. Domyślna architektura wyszukiwania zostanie wybrana zgodnie z wcześniej zdefiniowaną regułą decyzyjną, która priorytetowo traktuje oparcie wyników na dowodach, poprawność faktograficzną oraz zgodność z efektami uczenia się, przy jednoczesnym uwzględnieniu kosztów operacyjnych.

Słowa kluczowe: generowanie wspomagane wyszukiwaniem informacji, duże modele językowe, wyszukiwanie informacji, generowanie zadań laboratoryjnych, oparcie na dowodach, identyfikowalność źródeł, technologie edukacyjne, grafy wiedzy.

Table of Contents

Introduction 1

1 Theoretical Foundations of Retrieval-Augmented Generation 3

1.1 Large Language Models: Capabilities, Limitations, and Grounding 3

1.2 Information Retrieval and Closed-Domain Knowledge Sources 4

1.3 Retrieval-Augmented Generation: Components and Design Patterns 4

1.4 Retrieval Approaches: Lexical, Dense, Hybrid, and Reranking 5

1.5 Graph-Based, Adaptive, Agentic, and Multimodal RAG 6

1.6 Remarks and Conclusions 7

2 Principles of Corpus Preparation and Evaluation for RAG Systems 12

2.1 Corpus Boundaries, Source Authority, and Knowledge-Base Separation 12

2.2 Document Processing, Metadata, Segmentation, and Indexing 14

2.3 Benchmark Design and Evaluation Measures 15

2.4 Claim Validation, Reproducibility, and Architecture-Selection Criteria 17

2.5 Remarks and Conclusions 19

3 Design and Implementation of the Assignment Generation Application 20

3.1 Requirements, System Architecture, and Operating Systems Case Study 20

3.2 Corpus Preparation, Ingestion, Storage, and Metadata Management 23

3.3 Segmentation, Indexing, and Retrieval Architecture Implementation 25

3.4 Assignment Generation, Evidence Linking, Validation, and Web Application 27

3.5 Remarks and Conclusions 29

4 Experimental Evaluation and Architecture Selection 30

4.1 Experimental Protocol, Held-Out Benchmark, and Assessment Procedure 30

4.2 Retrieval, Evidence-Grounding, and Assignment-Quality Results 33

4.3 Claim-Validation, Unsupported-Request, and Conflict-Handling Results 36

4.4 Operational Evaluation and Comparative Analysis 38

4.5 Remarks and Conclusions 39

Summary 40

Bibliography 41

List of Figures 44

List of Tables 45

Appendices 46

Introduction

Generative artificial intelligence creates new opportunities to support the preparation of educational materials. Large language models can generate explanations, exercises, laboratory tasks, and assessment-related content from natural-language instructions. However, fluent output does not guarantee that the generated material is factually correct, consistent with a particular course, appropriate for the intended student group, or supported by approved teaching materials. A generated laboratory assignment may require tools that have not been introduced, conflict with course rules, omit essential prerequisites, or contain requirements that cannot be justified by the course documentation.

Retrieval-Augmented Generation (RAG) supplements a language model with information retrieved from an external collection of documents. Rather than relying only on knowledge encoded in model parameters, a RAG system retrieves passages relevant to a request and provides them as context for generation. This can improve the connection between generated content and a defined knowledge source. Nevertheless, retrieval alone does not ensure that the retrieved information is authoritative, complete, correctly interpreted, or sufficient to support every generated claim. In educational applications, traceability, evidence validation, handling of conflicting information, and explicit reporting of missing evidence are therefore essential.

The objective of this thesis is to design, implement, and evaluate a configurable web application that supports the creation of laboratory assignments for academic courses using Retrieval-Augmented Generation. The application will generate structured, evidence-grounded assignments from user-provided course materials according to configurable pedagogical requirements. The instructor will be able to specify a topic, learning outcomes, difficulty level, course stage, target competencies, expected duration, and assignment format. The generated assignment will contain a title, prerequisites, objectives, environment and tools, tasks, submission requirements, assessment rubric, and source evidence.

The application will use a closed corpus and will not use web search. The corpus will contain two separate local knowledge bases. The primary knowledge base will contain authoritative materials for the target course. A supplementary knowledge base will contain approved materials from related or earlier courses. This distinction is important because supplementary materials can be useful for background explanations, prerequisites, hints, and optional extensions, but they must not override primary materials when defining mandatory technical requirements, course-specific tools, learning outcomes, or assessment criteria.

Operating Systems course materials will be used as a case study. The work will implement persistent storage for source documents, extracted text, metadata, embeddings, indexes, generated assignments, claim-to-evidence links, benchmark requests, experiment configurations, execution logs, and evaluation results. The ingestion process will support PDF, HTML, TXT, and Markdown documents. Extracted fragments will retain source identifiers and retrievable excerpts, while metadata will include available information such as knowledge-base type, course, document type, topic, version, learning outcomes, difficulty level, and authority level.

The comparative part of the thesis will evaluate four retrieval architectures: baseline vector RAG, Hybrid RAG, LightRAG, and GraphRAG. The architectures will be compared using the same corpus, generation model, prompt template, output schema, validation procedure, held-out benchmark, and evaluation protocol. Unless explicitly included as an experimental factor, the embedding model, metadata policy, retrieved-context budget, and evaluation environment will remain constant. This design is intended to ensure that observed differences can be attributed primarily to the retrieval architecture rather than to unrelated changes in the experimental configuration.

The study is guided by the following research questions:

1. How do baseline vector RAG, Hybrid RAG, LightRAG, and GraphRAG compare in retrieval relevance, evidence coverage, and citation correctness when generating laboratory assignments from a closed Operating Systems course corpus?

2. How do the compared architectures differ in factual correctness, learning-outcome alignment, expert-rated educational usefulness, and handling of unsupported or conflicting evidence?

3. How accurately can the proposed evidence-validation mechanism classify generated claims as supported, contradicted, or unsupported?

4. What are the operational trade-offs among the evaluated architectures with respect to latency, indexing time, storage use, token usage, and estimated inference cost?

5. Which architecture best satisfies the predefined decision rule that prioritizes grounding, factual correctness, and learning-outcome alignment while considering operational cost?

To answer these questions, the corpus will first be documented and prepared. Before implementation or tuning of the compared architectures, the work will define evaluation measures, an expert-assessment rubric, and a held-out benchmark containing at least 60 requests. The benchmark will include supported single-source requests, multi-source requests, requests requiring permitted supplementary materials, unsupported requests, conflicting-source requests, and requests representing different difficulty levels or course stages. The benchmark will not be used for prompt development, configuration tuning, or architecture selection.

The intended contributions of the thesis are an evidence-grounded workflow for laboratory assignment generation from closed course-specific corpora; a configurable instructor-facing web application; a method for linking generated claims and mandatory requirements to source evidence; a reproducible benchmark and evaluation protocol; and a controlled comparison of four RAG architectures in the Operating Systems case study. The scope is limited to approved course materials and does not aim to replace instructor judgment. The final instructor remains responsible for reviewing generated assignments and approving their use.

The remainder of the thesis is organized as follows. Chapter 1 presents the theoretical foundations of large language models, information retrieval, and Retrieval-Augmented Generation. It discusses lexical, dense, hybrid, graph-based, adaptive, agentic, and multimodal approaches, together with grounding, provenance, conflict handling, and abstention. Chapter 2 introduces the theoretical principles of corpus preparation, benchmark construction, evaluation, validation, reproducibility, and architecture selection. Chapter 3 describes the Operating Systems case study and the design and implementation of the application. Chapter 4 presents the experimental procedure, evaluation results, comparative analysis, and selection of the default architecture. The summary synthesizes the findings, limitations, and possible directions for future work.

Chapter 1

Theoretical Foundations of Retrieval-Augmented Generation

This chapter presents the theoretical foundations required for the design and evaluation of Retrieval-Augmented Generation systems. It introduces the role and limitations of large language models, core information-retrieval concepts, and the components of a RAG pipeline. It then reviews the principal retrieval approaches, including lexical, dense, hybrid, graph-based, adaptive, agentic, and multimodal methods. The final section discusses provenance, evidence validation, conflict handling, and abstention.

The chapter is intentionally general. It does not describe the Operating Systems corpus, selected software tools, configuration values, or experimental results. Those application-specific aspects are presented in later chapters.

1.1 Large Language Models: Capabilities, Limitations, and Grounding

Large language models (LLMs) are neural networks that process and generate natural language. Most contemporary LLMs are based on the Transformer architecture, which uses self-attention mechanisms to model relationships among tokens in an input sequence [1]. During pretraining, a model learns statistical regularities from large text collections. This enables it to perform tasks such as question answering, summarization, translation, code generation, classification, and structured text generation.

LLMs can support the preparation of educational content because they can transform high-level instructions into coherent explanations, exercises, laboratory tasks, and assessment-related material. However, their output should not be treated as an authoritative representation of a specific course. The knowledge encoded in model parameters may be incomplete, outdated, inconsistent, or unrelated to the terminology, tools, learning outcomes, and assessment rules used in a particular educational setting.

Language generation is probabilistic. Given an input, an autoregressive model estimates the probability of each subsequent token based on the preceding context. Consequently, different decoding settings or small changes in the prompt can lead to different outputs. Fluency is therefore not evidence that a generated statement is correct, appropriate, or supported by a source.

A major limitation of LLMs is hallucination: the generation of information that is unsupported, false, misleading, or fabricated. Hallucinations may include invented facts, unjustified technical requirements, incorrect interpretations of a source, or fabricated citations. Ji et al. [2] distinguish factual errors from faithfulness errors, where generated content is not supported by the provided context. Both types are relevant in educational applications because an assignment may be plausible in general while remaining inconsistent with approved course materials.

LLMs also have finite context windows. Supplying an entire document collection to a model is usually inefficient, costly, and unreliable. Relevant information may be obscured by unrelated content, and large inputs can exceed context limits. Grounding addresses these limitations by providing selected external evidence to the model during generation. In text-based systems, grounding commonly relies on retrieved passages from a controlled corpus. The model then acts as a language generation component, while authoritative course-specific information originates from approved documents.

1.2 Information Retrieval and Closed-Domain Knowledge Sources

Information retrieval concerns the identification and ranking of documents or passages relevant to an information need. A user request is represented as a query, and the retrieval system searches a document collection to return a ranked set of candidate results. Retrieval can operate at document, section, passage, sentence, table, or other levels of granularity.

Relevance is not a single property. A retrieved passage may be topically related to a request but still fail to support a specific claim. It may also be semantically relevant while coming from a source that is not authoritative for the decision being made. In RAG systems, it is useful to distinguish topical relevance, semantic relevance, evidential relevance, and authority relevance. Topical relevance concerns the same subject as the request; semantic relevance conveys a similar meaning; evidential relevance supports, qualifies, or contradicts a specific claim; and authority relevance concerns whether the source is appropriate for the type of claim being generated.

The distinction between evidential and authority relevance is particularly important in educational applications. A background document may explain a technical concept accurately, but it may not be an appropriate source for a mandatory course requirement or assessment criterion.

Retrieval environments are often classified as open-domain or closed-domain. Open-domain systems can retrieve from broad collections such as the public web. They can provide diverse and current information, but they also introduce source quality, volatility, licensing, reproducibility, and governance challenges. Search results can change over time and may include unreviewed or unsuitable sources.

Closed-domain retrieval restricts search to a predefined and controlled corpus. The corpus may consist of institutional documents, technical manuals, legal documents, scientific literature, or course materials. This approach supports repeatability because the collection can be documented, versioned, and frozen for evaluation. It also enables explicit inclusion rules, source policies, metadata standards, and authority hierarchies. A closed corpus does not guarantee correctness or completeness, but it makes the evidence boundary explicit.

Documents are typically divided into smaller fragments before indexing. Passage retrieval improves precision and reduces the amount of text supplied to the generator. However, small fragments can omit necessary conditions or detach a requirement from its explanation. Larger fragments preserve context but can reduce precision and consume more of the available context budget. Segmentation is therefore a central retrieval design decision.

Metadata complements text-based retrieval. It may describe the document type, version, author, topic, course, source role, authority level, or learning outcomes. Metadata can support filtering and ranking, but it does not replace semantic evidence. A retrieval system should use both content relevance and source properties when selecting evidence for generation.

1.3 Retrieval-Augmented Generation: Components and Design Patterns

Retrieval-Augmented Generation combines an information-retrieval component with a language-generation model. Instead of relying only on knowledge stored in model parameters, the system retrieves relevant external information and supplies it as context to the generator. The original RAG formulation proposed by Lewis et al. [3] combines a dense retriever with a sequence-to-sequence model and treats retrieved documents as latent variables in the generation process.

In a simplified formulation, the probability of a generated output can be approximated by marginalizing over the top-ranked retrieved documents:

$$P(y \mid x) \approx \sum_{z \in \text{top-}k(x)} P_\eta(z \mid x) P_\theta(y \mid x, z). \tag{1.1}$$

where x is the input query, y is the generated response, z is a retrieved document or passage, P_η represents the retriever, and P_θ represents the generator. This formulation should be distinguished from many practical RAG systems, which use independently configured retrieval and generation components rather than jointly trained models.

A modular RAG system generally consists of document acquisition and corpus definition; parsing, text extraction, and normalization; segmentation into passages or other retrieval units; metadata assignment and index construction; query processing and filtering; candidate retrieval and reranking; context selection within a token budget; structured or free-text generation; and validation of output structure, source references, and evidence.

The simplest form, often called *naïve RAG*, retrieves a fixed number of top-ranked passages and inserts them into the generation prompt. *Advanced RAG* adds query rewriting, metadata filters, hybrid retrieval, rank fusion, reranking, contextual compression, and structured prompts. *Modular RAG* separates functions such as retrieval, ranking, generation, validation, routing, and memory, which supports controlled experimentation and error analysis.

Iterative RAG performs several retrieval and generation cycles. *Corrective RAG* assesses the retrieved context and can reformulate a query, use an alternative retrieval route, retrieve additional material, or abstain when evidence is insufficient [4]. *Self-reflective RAG* likewise introduces mechanisms for evaluating the relevance of retrieved evidence and the support for a generated answer [5]. RAG is a grounding mechanism rather than a correctness guarantee: evidence may not be retrieved, retrieved passages may be incomplete or contradictory, and the generator may misinterpret the available context. Retrieval quality and output grounding must therefore be evaluated separately.

1.4 Retrieval Approaches: Lexical, Dense, Hybrid, and Reranking

Retrieval methods differ in how they represent queries and documents, estimate relevance, and search an indexed corpus. The principal families include lexical retrieval, dense-vector retrieval, sparse neural retrieval, late-interaction retrieval, hybrid retrieval, and reranking.

Lexical Retrieval

Lexical retrieval represents documents using observed words or terms. A common data structure is the inverted index, which maps each term to the documents or passages that contain it. Boolean retrieval is a basic lexical method in which logical operators such as `AND`, `OR`, and `NOT` define strict inclusion and exclusion criteria.

BM25 is a widely used lexical ranking function. It estimates relevance from term frequency, inverse document frequency, and document-length normalization [6]:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \frac{f(t, d)(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \tag{1.2}$$

Here, $f(t, d)$ is the frequency of term t in document d, $|d|$ is document length, avgdl is average document length, and k_1 and b control term-frequency saturation and length normalization. Lexical retrieval performs well for exact terminology, identifiers, commands, version numbers, names, and phrases that occur directly in source documents. Its main limitation is vocabulary mismatch.

Dense, Sparse, and Late-Interaction Retrieval

Dense retrieval represents queries and passages as continuous embedding vectors. Similarity is often measured using cosine similarity:

$$\mathrm{sim}_{\cos}(q, d) = \frac{q \cdot d}{\|q\|\|d\|}. \tag{1.3}$$

A vector index returns passages nearest to the query embedding, often using approximate nearest-neighbour search. Dense retrieval can retrieve semantically similar passages even when they do not share exact words with the query. Dense Passage Retrieval is a prominent example [7]. However, embedding similarity does not prove evidential support, and rare technical terms can be handled less reliably than by lexical retrieval.

Sparse neural retrieval uses learned term weighting or expansion while retaining sparse representations compatible with inverted indexes. SPLADE is a representative approach [8]. Late-interaction retrieval retains token-level representations for queries and documents and computes fine-grained interactions during retrieval. ColBERT is a well-known example [9]. These methods can improve semantic matching but may require more storage or computation than standard lexical or dense retrieval.

Hybrid Retrieval and Reranking

Hybrid retrieval combines two or more retrieval signals, most commonly lexical and dense-vector rankings. The motivation is that lexical retrieval is effective for exact terminology, while dense retrieval is effective for semantic similarity and paraphrase. A common rank-fusion approach is Reciprocal Rank Fusion (RRF):

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}, \tag{1.4}$$

where R is the set of rankings, $\mathrm{rank}_r(d)$ is the rank of document d in ranking r, and k is a smoothing constant [10]. Rank-based fusion is useful because lexical and vector scores often have different scales.

Reranking is a second-stage process in which a stronger model reorders a smaller candidate set returned by the initial retriever. Cross-encoders can process a query and candidate passage jointly, often improving precision at the highest ranks. However, reranking increases latency and computational cost. The BEIR benchmark demonstrates that retrieval effectiveness, latency, index size, and zero-shot generalization can vary substantially across datasets and models [11]. Retrieval methods should therefore be evaluated against the target corpus and task.

1.5 Graph-Based, Adaptive, Agentic, and Multimodal RAG

Text-based retrieval can be extended with structured representations, hierarchical document models, adaptive control mechanisms, agent-based planning, and multimodal processing.

Graph-Based RAG

Graph-based RAG represents entities and their relationships explicitly. A knowledge graph can be described as

$$G = (V, E), \tag{1.5}$$

where V is a set of nodes and E is a set of edges. Nodes may represent concepts, documents, tools, tasks, learning outcomes, or other entities. Edges may represent relations such as *prerequisite-of*, *described-in*, *depends-on*, or *assessed-by*.

Graph structures can support multi-hop retrieval, relationship discovery, concept clustering, and corpus-level analysis. Graph construction requires entity extraction, relation extraction, entity resolution, and source linking. These processes can introduce errors: entities may be merged incorrectly, relations may be incomplete or reversed, and graph connections may omit conditions stated in the source text. Graph nodes and edges should therefore remain linked to their source passages. Graph structure is a retrieval signal, not a substitute for inspectable evidence.

LightRAG combines text-unit retrieval with an entity-relation graph and supports local entity-focused retrieval and broader relationship-oriented retrieval [12]. GraphRAG constructs an entity graph, detects communities, and creates community reports to support local search and corpus-wide synthesis [13]. These approaches are related but not identical, and their particular configurations should be documented precisely in empirical work.

Hierarchical, Adaptive, and Agentic RAG

Hierarchical RAG organizes information at several levels of detail, for example as sections, chunks, parent passages, and summaries. Parent-child retrieval uses small units for search precision while returning a larger parent section to preserve context. Recursive-summary methods create higher-level representations from lower-level passages. RAPTOR is an example of a method that builds a hierarchy of recursive summaries and retrieves at different abstraction levels [14]. Summaries can omit qualifications or introduce abstraction errors, so technical claims should remain traceable to original source excerpts.

Adaptive RAG selects retrieval strategies dynamically according to query properties, estimated complexity, source availability, or the quality of initial results. Corrective RAG can reformulate a query, retrieve additional material, choose another source, or abstain when evidence is inadequate [4]. These mechanisms avoid applying the same retrieval strategy to every request but increase system complexity, latency, and potential nondeterminism.

Agentic RAG combines retrieval with planning, tool use, task decomposition, and iteration. An agent can formulate subquestions, choose retrieval tools, inspect evidence, revise a plan, and synthesize a final output. This flexibility may help with complex information-seeking tasks, but it also increases token use, latency, implementation complexity, and reproducibility risks. Evidence-grounded agentic systems should operate within explicit source boundaries and preserve links from final claims to original evidence.

Multimodal RAG

Multimodal RAG retrieves and uses information from more than plain text. Educational materials can include tables, diagrams, slides, screenshots, source code, equations, audio, and video. Multimodal systems may combine optical character recognition, table extraction, document-layout analysis, vision-language models, and modality-specific embeddings. They can retrieve text and images jointly or transform non-textual content into textual representations before indexing.

Multimodal processing introduces additional risks. Optical character recognition may corrupt source code or mathematical notation, table extraction may lose structural relations, and diagram interpretation can be ambiguous. Such systems should preserve original source locations and modalities so that generated interpretations can be inspected and verified.

1.6 Remarks and Conclusions

The purpose of evidence-grounded RAG is not only to retrieve topically related content, but also to enable generated claims to be traced to appropriate and sufficient evidence. Provenance describes the origin, context, and transformation history of information. In a RAG system, provenance should connect generated claims to retrieved passages, source documents, document versions, metadata, and, where relevant, graph relations or summaries derived from source text.

Source traceability can be maintained at document, fragment, metadata, claim, and transformation levels. Document-level provenance identifies the original source; fragment-level provenance identifies the retrieved passage or excerpt; metadata-level provenance identifies properties such as version, source role, and authority; claim-level provenance links a generated statement to evidence; and transformation provenance records parsing, segmentation, embedding, summarization, or graph construction steps that influence retrieval.

The presence of a citation does not prove that a claim is grounded. A cited source may exist while failing to support the associated statement. Evidence validation should therefore distinguish

three outcomes: *supported*, where evidence directly supports a claim; *contradicted*, where evidence conflicts with it; and *unsupported*, where evidence neither supports nor contradicts it sufficiently. Topical similarity alone is insufficient for technical requirements, mandatory tasks, or assessment criteria.

Conflicting evidence can arise because documents refer to different versions, describe optional alternatives, apply to different contexts, or contain genuine inconsistencies. Conflict handling should consider source authority, version, scope, and recency. The system should not silently select one source or combine incompatible instructions. If a conflict cannot be resolved by the source policy, it should be reported for human review.

Abstention is therefore a necessary RAG behavior. When required evidence is missing, contradictory, or invalid, the system should not present its output as complete and verified. Instead, it should provide a structured response identifying the unsupported requirement, conflicting sources, or missing information. Automated validation can verify schema compliance, mandatory sections, source-reference validity, metadata policies, and some forms of semantic support, but it cannot fully replace expert judgment about educational workload, pedagogical sequencing, accessibility, fairness, or course-specific suitability.

In conclusion, LLMs can generate coherent text but cannot serve as reliable authorities for course-specific content without external grounding. RAG provides a framework for combining generation with retrieval from controlled knowledge sources. Lexical, dense, sparse, late-interaction, hybrid, graph-based, hierarchical, adaptive, agentic, and multimodal approaches offer different ways of representing and retrieving information. Their suitability depends on the corpus, task, source policies, evidence requirements, and operational constraints. Chapter 2 presents the methodological principles for corpus preparation, knowledge-base separation, benchmark design, retrieval and generation evaluation, claim validation, reproducibility, and multi-criteria architecture selection.

Chapter 2

Principles of Corpus Preparation and Evaluation for RAG Systems

2.1 Corpus Boundaries, Source Authority, and Knowledge-Base Separation

To be completed. This chapter will define generic principles for controlled corpora, primary and supplementary source roles, document processing, benchmark construction, evaluation, validation, reproducibility, and architecture-selection criteria.

2.2 Document Processing, Metadata, Segmentation, and Indexing

To be completed.

2.3 Benchmark Design and Evaluation Measures

To be completed.

2.4 Claim Validation, Reproducibility, and Architecture-Selection Criteria

To be completed.

2.5 Remarks and Conclusions

To be completed.

Chapter 3

Design and Implementation of the Assignment Generation Application

3.1 Requirements, System Architecture, and Operating Systems Case Study

To be completed.

3.2 Corpus Preparation, Ingestion, Storage, and Metadata Management

To be completed.

3.3 Segmentation, Indexing, and Retrieval Architecture Implementation

To be completed.

3.4 Assignment Generation, Evidence Linking, Validation, and Web Application

To be completed.

3.5 Remarks and Conclusions

To be completed.

Chapter 4

Experimental Evaluation and Architecture Selection

4.1 Experimental Protocol, Held-Out Benchmark, and Assessment Procedure

To be completed.

4.2 Retrieval, Evidence-Grounding, and Assignment-Quality Results

To be completed.

4.3 Claim-Validation, Unsupported-Request, and Conflict-Handling Results

To be completed.

4.4 Operational Evaluation and Comparative Analysis

To be completed.

4.5 Remarks and Conclusions

To be completed.

Summary

To be completed. This section will synthesize the results, answer the research questions, state limitations, and identify directions for future work.

Bibliography

[1] A. Vaswani, N. Shazeer, N. Parmar, *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[2] Z. Ji, N. Lee, R. Frieske, *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, 2023.

[3] P. Lewis, E. Perez, A. Piktus, *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, 2020.

[4] S.-Q. Yan, J.-C. Gu, Y. Zhu, *et al.*, "Corrective retrieval augmented generation," *arXiv preprint arXiv:2401.15884*, 2024.

[5] A. Asai, Z. Wu, Y. Wang, *et al.*, "Self-rag: Learning to retrieve, generate, and critique through self-reflection," *International Conference on Learning Representations*, 2024.

[6] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: Bm25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[7] V. Karpukhin, B. Oguz, S. Min, *et al.*, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020.

[8] T. Formal, B. Piwowarski, and S. Clinchant, "Splade: Sparse lexical and expansion model for first stage ranking," in *Proceedings of SIGIR*, 2021.

[9] O. Khattab and M. Zaharia, "Colbert: Efficient and effective passage search via contextualized late interaction over bert," in *Proceedings of SIGIR*, 2020.

[10] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of SIGIR*, 2009.

[11] N. Thakur, N. Reimers, A. Daxenberger, and I. Gurevych, "Augmented sbert for domain-specific semantic search," in *Proceedings of NAACL*, 2021.

[12] Y. Guo *et al.*, "Lightrag: Simple and fast retrieval-augmented generation," *arXiv preprint arXiv:2410.05779*, 2024.

[13] D. Edge *et al.*, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[14] P. Sarthi *et al.*, "Raptor: Recursive abstractive processing for tree-organized retrieval," *International Conference on Learning Representations*, 2024.